User Interface Design Document

Reference guide based on supplied React component code and customer/admin page text.

LaundryCare

1. Design summary

The application combines a React/Vite landing and order-booking interface styled with Tailwind utility classes, and Django-rendered pages for customer account workflows and staff/admin reporting. The UI uses laundry-service language, service cards, forms, status information and tables.

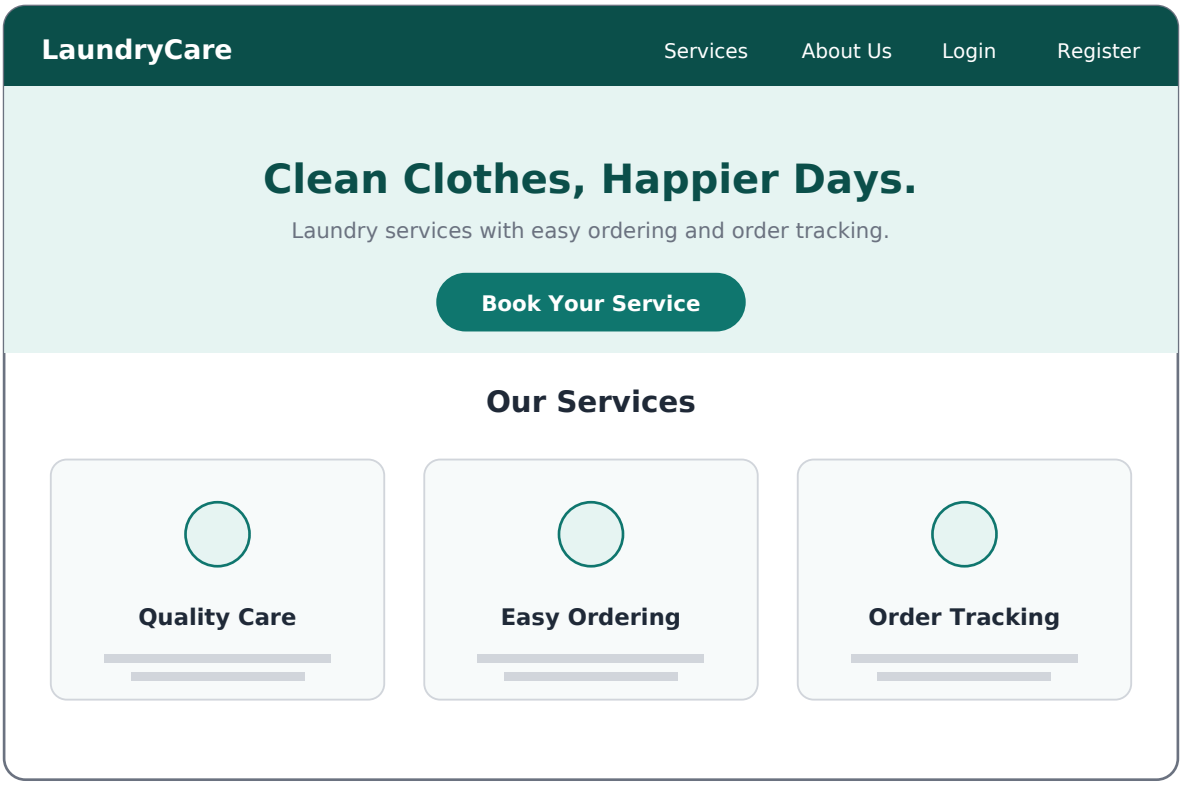
Observed visual conventions

Area	Observed implementation
Landing page	LaundryCare header, hero message “Clean Clothes, Happier Days.”, service cards, calls to action, feature highlights and footer.
Booking form	Address and service selectors, garment type, quantity (1–100), expected date, estimated total, validation/error messages and Place Order button.
Customer dashboard	Sidebar/navigation links for orders, profile, notifications, payments and deliveries; welcome area and order-booking action.
Order tracking	Order number, order status, expected date, delivery status and eight processing stages.
Payments	Per-order total, discount, advance, pending amount, payment status and payment date.
Notifications	Order Received and Delivery Update messages, order reference, timestamp, read/unread state and Mark as Read action.
Admin portal	Dashboard, All Orders, Daily Orders Report, Revenue Report and Logout navigation.
Admin order list	Order ID, customer, order date, expected date, status and total amount.
Reports	Daily order date/count table and revenue totals for amount, discount, advance received and pending amount.

2. Schematic wireframes

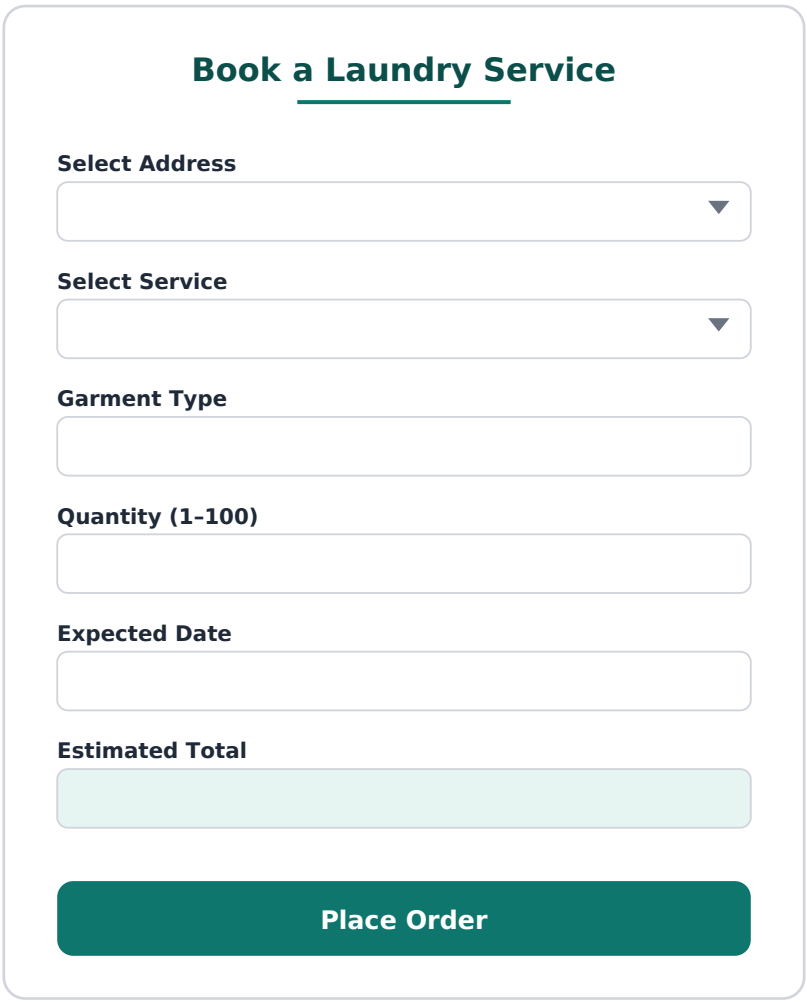
These are simplified layout wireframes created from the supplied code/text; they are not screenshots or pixel-perfect replicas.

Landing page

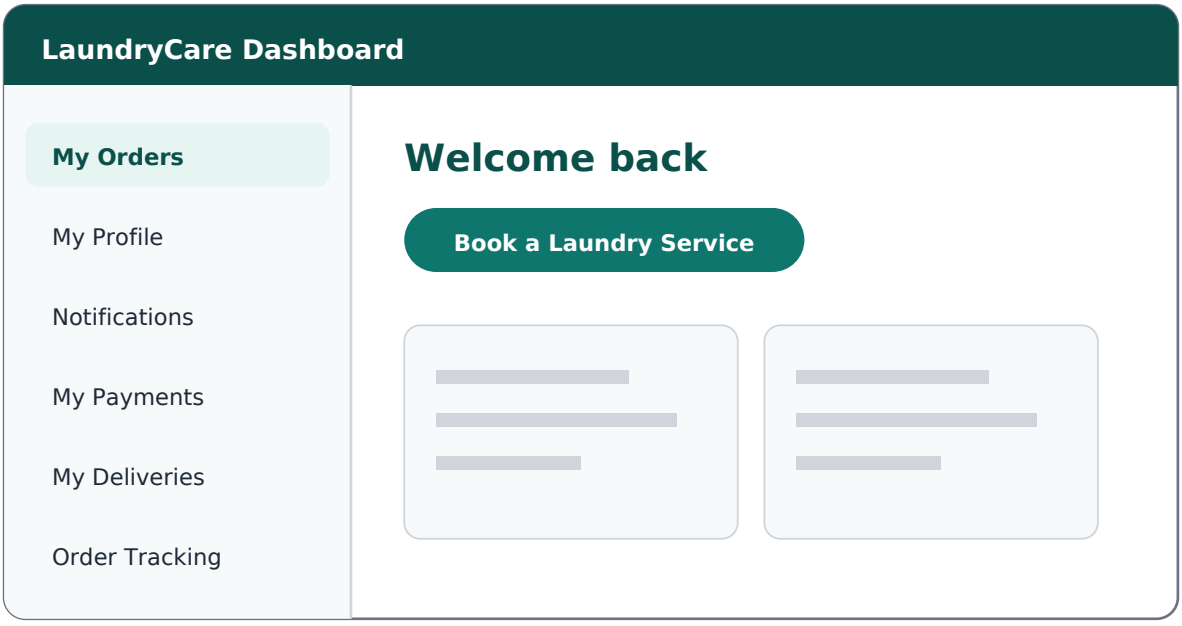


Schematic wireframe based on supplied App.jsx; not a screenshot.

Order booking page



Customer dashboard concept



Schematic wireframe based on supplied dashboard text.

3. Responsive design review

Evidence in supplied React code: the services section uses a responsive grid (one column by default, two at small breakpoints and three at large breakpoints); the booking form uses a constrained max-width, horizontal padding and responsive card padding. Inputs and primary action buttons are full-width. These choices support narrow screens.

Known mobile navigation concern

> In the supplied App.jsx, Services and About Us navigation links are hidden below the small breakpoint, and no hamburger/menu toggle is shown in that component. Confirm the intended mobile navigation in a real browser before calling the interface fully mobile-tested.

4. Functional interaction notes

The booking component loads services and saved addresses through JSON endpoints with credentials included, calculates an estimated total from selected service price × quantity, obtains a CSRF token before POSTing an order request, displays loading/error states, and redirects to the dashboard after a successful response.

The supplied view logic indicates booking creates an order, order items, processing stages, a scheduled delivery, a pending payment record and an order-received notification. Payment gateway processing is not documented as implemented.

5. Accessibility and verification checklist

Check	Status from supplied evidence
Form labels	Visible labels are associated with form controls using htmlFor/id in OrderBooking.jsx.
Validation	Quantity limits and required fields are represented in the form; backend also validates quantity and expected date.
Feedback	Loading and error feedback is rendered; errors use role=alert and loading uses role=status in the booking component.
Mobile navigation	Needs browser verification; no hamburger menu is present in the supplied App.jsx excerpt.
Mobile layout	Responsive classes are present, but actual device/browser testing remains outstanding.
Payment gateway	Not confirmed in the supplied code; do not represent as implemented.